

PHP CRUD Car Dealership

José Manuel Sánchez Álvarez

Contents

1	Car Dealership CRUD in PHP	1
1.1	Database	2
1.1.1	Table: CarBrand	2
1.1.2	Table: Seller	2
1.1.3	Table: Car	2
1.1.4	Table: Client	3
1.1.5	Table: Purchase	3
1.2	Notes on the Database Design	3
2	PHP and HTML Code	4
2.1	List ALL Cars	4
2.1.1	Step 1: Create the HTML Page	4
2.1.2	Step 2: Create the PHP Script	4
2.1.3	Explanation:	5
2.1.4	Note:	5
2.2	One car deatails	6
2.2.1	Step 1: Create the HTML Page for Car Selection	6
2.2.2	Step 2: Create the PHP Script for Displaying Car Details	6
2.2.3	Explanation:	7
2.2.4	Note:	7
2.3	Create a new car	8
2.3.1	Step 1: Create the HTML Form	8
2.3.2	Step 2: Create the PHP Script for Form Handling	8
2.3.3	Explanation	9
2.3.4	Note	10

1 Car Dealership CRUD in PHP

Creating a CRUD (Create, Read, Update, Delete) application for a car dealership in PHP involves designing a suitable database schema. Below, I'll outline a basic database structure with tables, attributes, types, constraints, keys, and relationships. This schema is designed to manage information about cars, brands, sellers, and clients.

1.1 Database

1.1.1 Table: CarBrand

- **Attributes:**
 - id (INT, primary key, auto-increment)
 - name (VARCHAR, unique, not null)
- **Primary Key:** id
- **Unique Key:** name
- **Purpose:** Stores different car brands.

1.1.2 Table: Seller

- **Attributes:**
 - id (INT, primary key, auto-increment)
 - name (VARCHAR, not null)
 - cars_sold (INT, default 0)
- **Primary Key:** id
- **Purpose:** Stores information about car sellers.

1.1.3 Table: Car

- **Attributes:**
 - id (INT, primary key, auto-increment)
 - model (VARCHAR, not null)
 - price (DECIMAL(10,2), not null)
 - condition (VARCHAR, not null, values like 'new', 'refurbished')
 - engine_features (TEXT, nullable)
 - gas_type (VARCHAR, nullable)
 - mileage (INT, nullable)
 - status (VARCHAR, nullable)
 - gears (INT, nullable)
 - license_plate (VARCHAR, nullable)
 - deposit_capacity (DECIMAL(5,2), nullable)
 - trunk_capacity (DECIMAL(5,2), nullable)
 - horse_power (INT, nullable)

- `brand_id` (INT, foreign key to CarBrand, not null)
- `seller_id` (INT, foreign key to Seller, not null)
- **Primary Key:** `id`
- **Foreign Keys:**
 - `brand_id` references CarBrand(`id`)
 - `seller_id` references Seller(`id`)
- **Purpose:** Stores details about each car.

1.1.4 Table: Client

- **Attributes:**
 - `id` (INT, primary key, auto-increment)
 - `name` (VARCHAR, not null)
- **Primary Key:** `id`
- **Purpose:** Stores client information.

1.1.5 Table: Purchase

- **Attributes:**
 - `id` (INT, primary key, auto-increment)
 - `client_id` (INT, foreign key to Client, not null)
 - `car_id` (INT, foreign key to Car, not null)
 - `date` (DATE, not null)
- **Primary Key:** `id`
- **Foreign Keys:**
 - `client_id` references Client(`id`)
 - `car_id` references Car(`id`)
- **Purpose:** Records the purchases of cars by clients.

1.2 Notes on the Database Design

- **Normalization:** The database design should be normalized to at least the third normal form (3NF) to reduce redundancy and improve data integrity.
- **Data Types:** Choose appropriate data types for each field. For instance, VARCHAR for strings, INT for integers, DECIMAL for prices and capacities, and DATE for dates.
- **Constraints:**

- NOT NULL constraints are used for mandatory fields.
- AUTO_INCREMENT is used for primary keys to create unique identifiers for each row.
- UNIQUE constraints ensure values like brand names are not duplicated.
- **Foreign Key Relationships:** These enforce referential integrity between tables. For example, each car is associated with a specific brand and seller.
- **Nullable Fields:** Fields like engine_features, gas_type, mileage, etc., are allowed to be NULL to accommodate varying levels of detail available for different cars.

This database structure provides a comprehensive foundation for a car dealership system, enabling efficient data storage and retrieval for a wide range of dealership operations.

2 PHP and HTML Code

2.1 List ALL Cars

To create a simple HTML page and its corresponding PHP script for listing all cars with their model, brand, and price in a car dealership project, you'll need to follow these steps. This example assumes you have a database with the necessary tables and data in place.

2.1.1 Step 1: Create the HTML Page

This page (list_cars.html) will be simple, primarily serving as a link to the PHP script that will display the car list.

```
<!DOCTYPE html>
<html>
<head>
  <title>Car Dealership - Car List</title>
</head>
<body>
  <h1>Car List</h1>
  <p>
    <a href="list_cars.php">View All Cars</a>
  </p>
</body>
</html>
```

2.1.2 Step 2: Create the PHP Script

This PHP script (list_cars.php) will connect to the database, retrieve the list of cars, and display the model, brand, and price. We'll use PDO for database interaction with prepared statements.

```
<?php
$host = 'localhost';
$db   = 'car_dealership';
```

```

$user = 'root';          // Replace with your database username
$pass = '';              // Replace with your database password
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    throw new \PDOException($e->getMessage(), (int)$e->getCode());
}

$stmt = $pdo->prepare("SELECT Car.model, Car.price, CarBrand.name AS brand FROM C
$stmt->execute();

echo "<h1>Car List</h1>";
echo "<table border='1'><tr><th>Model</th><th>Brand</th><th>Price</th></tr>";
while ($row = $stmt->fetch()) {
    echo "<tr><td>".$row["model"]."</td><td>".$row["brand"]."</td><td>".$row["pri
}
echo "</table>";
?>

```

2.1.3 Explanation:

1. **Database Connection:** The script starts by setting up a PDO connection to the database.
2. **Prepared Statement:** A prepared statement is used to query the database. This query joins the Car and CarBrand tables to get the model, brand, and price of each car.
3. **Data Display:** The fetched data is displayed in an HTML table.

2.1.4 Note:

- Make sure to replace the \$host, \$db, \$user, and \$pass variables with your actual database host, database name, username, and password.
- The structure of the SQL query depends on your database schema. Adjust the field names and table names according to your actual database schema.
- Ensure that your PHP environment is correctly set up to connect to your MySQL database.

This setup provides a basic and secure way to list cars from your database using PHP and PDO with prepared statements.

2.2 One car details

To create an HTML page and a corresponding PHP script for showing the details of a specific car selected by its ID, you'll need to follow these steps. This example assumes you have a database with the necessary tables and data, including detailed information about each car.

2.2.1 Step 1: Create the HTML Page for Car Selection

This page (select_car.html) allows the user to enter the ID of the car they want to view.

```
<!DOCTYPE html>
<html>
<head>
    <title>Car Dealership - Select Car</title>
</head>
<body>
    <h1>Select a Car to View Details</h1>
    <form action="car_detail.php" method="get">
        Car ID: <input type="text" name="car_id">
        <input type="submit" value="View Details">
    </form>
</body>
</html>
```

2.2.2 Step 2: Create the PHP Script for Displaying Car Details

This PHP script (car_detail.php) will connect to the database, retrieve the details of the car with the given ID, and display them. We'll continue using PDO for database interaction with prepared statements.

```
<?php
$host = 'localhost';
$db   = 'car_dealership';
$user = 'root';          // Replace with your database username
$pass = '';               // Replace with your database password
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES  => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
```

```

        throw new \PDOException($e->getMessage(), (int)$e->getCode());
    }

$carId = $_GET['car_id'] ?? ''; // Retrieve car ID from URL parameter

if ($carId) {
    $stmt = $pdo->prepare("SELECT * FROM Car WHERE id = :carId");
    $stmt->bindParam(':carId', $carId, PDO::PARAM_INT);
    $stmt->execute();

    $car = $stmt->fetch();
    if ($car) {
        echo "<h1>Car Details</h1>";
        echo "<p>Model: " . htmlspecialchars($car["model"]) . "</p>";
        echo "<p>Brand: " . htmlspecialchars($car["brand"]) . "</p>";
        // Display other details like price, condition, engine features, etc.
    } else {
        echo "<p>Car not found.</p>";
    }
} else {
    echo "<p>No car selected.</p>";
}
?>

```

2.2.3 Explanation:

1. **Database Connection:** The script sets up a PDO connection to the database.
2. **Retrieving Car ID:** The car ID is obtained from the URL parameter (`$_GET['car_id']`).
3. **Query and Display Car Details:** A prepared statement is used to retrieve and display the details of the car with the provided ID. The data is sanitized using `htmlspecialchars` to prevent XSS attacks.

2.2.4 Note:

- Replace the database connection details (`$host`, `$db`, `$user`, `$pass`) with your actual database information.
- Ensure your table and column names in the SQL query match those in your database schema.
- The script assumes that the `Car` table contains all necessary columns like `model`, `brand`, `price`, `condition`, etc. Modify the script to include all relevant details as per your database schema.
- This setup provides a basic and secure way to display detailed information about a car from your database using PHP and PDO. For a more user-friendly interface, you might consider adding error handling for invalid IDs and styling the output with CSS.

2.3 Create a new car

Creating an HTML form and corresponding PHP script for inserting a new car with all its details into a database involves two main parts: designing the form and handling the form submission with PHP. Below is a basic example. This assumes you have a database structure ready to store these details.

2.3.1 Step 1: Create the HTML Form

Create an HTML page (add_car.html) for the input form. This form will include various fields for car details. Some fields will be marked as mandatory.

```
<!DOCTYPE html>
<html>
<head>
  <title>Add New Car</title>
</head>
<body>
  <h1>Add New Car</h1>
  <form action="insert_car.php" method="post">
    Model (required): <input type="text" name="model" required><br>
    Brand (required): <input type="text" name="brand" required><br>
    Price (required): <input type="number" name="price" required><br>
    Condition: <input type="text" name="condition"><br>
    Engine Features: <input type="text" name="engine_features"><br>
    Gas Type: <input type="text" name="gas_type"><br>
    Mileage: <input type="number" name="mileage"><br>
    Status (new/refurbished): <input type="text" name="status"><br>
    Gears: <input type="number" name="gears"><br>
    License Plate: <input type="text" name="license_plate"><br>
    Deposit Capacity: <input type="number" name="deposit_capacity"><br>
    Trunk Capacity: <input type="number" name="trunk_capacity"><br>
    Horse Power: <input type="number" name="horse_power"><br>
    <input type="submit" value="Add Car">
  </form>
</body>
</html>
```

2.3.2 Step 2: Create the PHP Script for Form Handling

Create a PHP script (insert_car.php) to handle the form submission. This script will insert the new car details into your database.

```
<?php
$host = 'localhost';
$db    = 'car_dealership';
$user = 'root';          // Replace with your database username
```

```

$pass = ''; // Replace with your database password
$charset = 'utf8mb4';

$dsn = "mysql:host=$host;dbname=$db;charset=$charset";
$options = [
    PDO::ATTR_ERRMODE            => PDO::ERRMODE_EXCEPTION,
    PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
    PDO::ATTR_EMULATE_PREPARES   => false,
];

try {
    $pdo = new PDO($dsn, $user, $pass, $options);
} catch (\PDOException $e) {
    throw new \PDOException($e->getMessage(), (int)$e->getCode());
}

if ($_SERVER["REQUEST_METHOD"] == "POST") {
    $stmt = $pdo->prepare("INSERT INTO Car (model, brand, price, condition, engine

    $stmt->execute([
        $_POST['model'],
        $_POST['brand'],
        $_POST['price'],
        $_POST['condition'] ?? null,
        $_POST['engine_features'] ?? null,
        $_POST['gas_type'] ?? null,
        $_POST['mileage'] ?? null,
        $_POST['status'] ?? null,
        $_POST['gears'] ?? null,
        $_POST['license_plate'] ?? null,
        $_POST['deposit_capacity'] ?? null,
        $_POST['trunk_capacity'] ?? null,
        $_POST['horse_power'] ?? null
    ]);

    echo "New car added successfully.";
}
?>

```

2.3.3 Explanation

- The HTML form includes various fields for entering car details. The required attribute is added to mandatory fields.
- The PHP script connects to the database and uses a prepared statement to insert the car data. It uses the POST method to retrieve data from the form.

- Null coalescing operators (??) are used for optional fields to handle cases where no data is entered.
- Proper error handling should be implemented for a production environment (e.g., try-catch blocks, validation of input data).

2.3.4 Note

- Replace the `$host`, `$db`, `$user`, and `$pass` variables with your actual database host, database name, username, and password.
- The SQL query in the prepared statement should match your database's table structure. Adjust the field names and table name according to your database schema.
- This example does not include client-side or server-side validation for the sake of brevity. In a real-world application, you should validate and sanitize all inputs to ensure data integrity and security.
- The form assumes that the user knows the Car ID they wish to update. In a practical scenario, you might have a selection process or a search feature to find the car before updating its details.